
Universidad de Guayaquil
Facultad de Ciencias Matemáticas y Físicas
Carrera de Software

Manual técnico

Instalación, operación y réplica del entorno

Prototipo web de inteligencia de negocios asistido por IA

Autoras	Emily Dayan Robles Alvarado y Lesly Samay Velásquez Anchundia
Versión del manual	1.1.0
Línea base	Ramas <code>develop</code> y <code>main</code>
Repositorio	LFXAS/prototipo-bi-ia
Clasificación	Uso académico y técnico - repositorio público
Fecha	24 de agosto de 2026

Índice

1. Propósito y audiencia	1
2. Arquitectura operativa	1
2.1. Servicios	1
2.2. Estructura del repositorio	1
3. Requisitos previos	2
3.1. Hardware recomendado	2
3.2. Software requerido	2
4. Configuración segura	2
4.1. Archivo de entorno	2
5. Instalación de desarrollo en este computador	3
6. Instalación en otro computador desde el código	4
6.1. Obtener el repositorio	4
6.2. Preparar secretos	4
6.3. Construir y levantar	4
6.4. Confirmar el estado	4
6.5. Abrir la aplicación	4
7. Ejecución desde imágenes privadas de GHCR	5
7.1. Autenticarse en GHCR	5
7.2. Configurar la entrega	6
7.3. Descargar e iniciar	6
8. Comprobación técnica	6
8.1. Comprobación rápida	6
8.2. Comprobación completa	6
8.3. Contrato OpenAPI	7
8.4. Validación de AdventureWorks	7
9. Operación diaria	8
10. Persistencia y reinicio de datos	8
10.1. Detener sin borrar	8

10.2. Reconstruir desde cero	8
10.3. Qué se versiona	8
11.Actualización y reversión	8
11.1. Entorno construido desde código	8
11.2. Entorno basado en GHCR	9
12.Flujo Git y CI/CD	9
12.1. Desarrollo por rama	9
12.2. Integración continua	9
12.3. Entrega continua	10
13.Seguridad operativa	10
14.Diagnóstico de fallos	10
14.1. Registros útiles	11
15.Lista de comprobación para una demostración	11
16.Mantenimiento de la documentación	12
17.Responsabilidades y escalamiento	12
A. Referencia rápida de URLs y puertos	12
B. Recuperación mínima	13
C. Control del manual	13

1. Propósito y audiencia

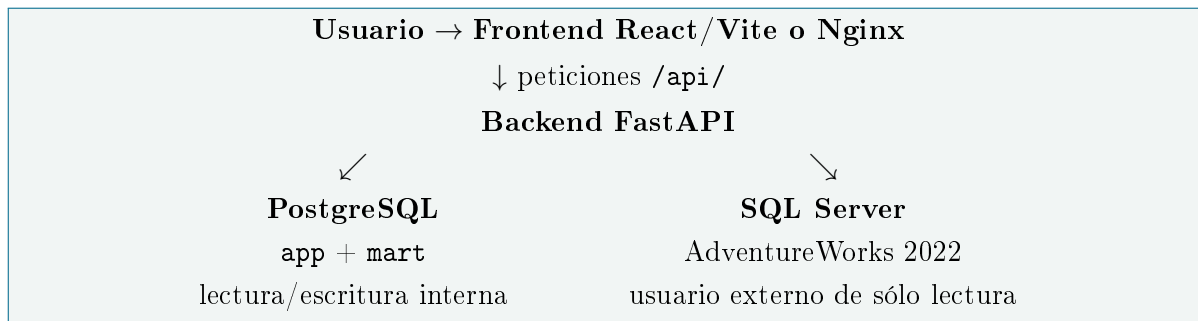
Este manual permite instalar, levantar, comprobar, mantener y replicar el ambiente técnico del prototipo. Está dirigido a las autoras, docentes revisores, colaboradores autorizados y personal técnico que necesite reproducir la demostración.

El documento cubre dos modalidades:

1. **Desarrollo desde el código:** clona el repositorio y construye las imágenes localmente.
2. **Ejecución de una entrega:** descarga imágenes privadas de GitHub Container Registry (GHCR) y no compila el código.

La versión actual contiene infraestructura y salud técnica. El RBAC y los módulos de negocio todavía no están implementados.

2. Arquitectura operativa



2.1. Servicios

Servicio	Responsabilidad	Puerto desarrollo	Persistencia
frontend	Interfaz React y proxy a la API	5173	Código y dependencias
backend	API FastAPI, configuración y salud	8000	Código montado
postgres	Configuración interna y futuro datamart	55432	Volumen <code>postgres_data</code>
sqlserver	Fuente AdventureWorks externa	51433	Volumen <code>sqlserver_data</code>

Compose crea la red y los volúmenes bajo el nombre lógico `bi-ia-prototype`. No usa los contenedores PostgreSQL o SQL Server que ya existan en el computador.

2.2. Estructura del repositorio

<code>backend/</code>	FastAPI, pruebas y migraciones
<code>frontend/</code>	React, Vite, pruebas y Nginx
<code>infra/postgres/</code>	imagen e inicializacion PostgreSQL
<code>infra/sqlserver/</code>	imagen y restauracion AdventureWorks
<code>infra/docs/</code>	imagen reproducible de LaTeX
<code>scripts/</code>	arranque, espera y verificacion
<code>docs/</code>	arquitectura, guias y evidencias

<code>.github/workflows/</code>	integración y entrega continuas
<code>compose.yaml</code>	entorno de desarrollo autocontenido
<code>compose.release.yaml</code>	entorno con imágenes de GHCR
<code>Makefile</code>	comandos operativos abreviados

3. Requisitos previos

3.1. Hardware recomendado

- Procesador de 64 bits con virtualización habilitada.
- 8 GB de memoria disponibles para Docker; 6 GB es el mínimo práctico.
- 15 GB libres para imágenes, capas, respaldo y volúmenes.
- Conexión a Internet durante la primera construcción y restauración.

3.2. Software requerido

- Docker Engine 24 o superior y Docker Compose v2; Docker Desktop es válido.
- Git para clonar y actualizar el repositorio.
- Acceso al repositorio público; permiso de escritura sólo para colaboradores autorizados.
- Para consumir imágenes privadas: credenciales GHCR con `read:packages`.

No se requiere instalar Python, Node.js, PostgreSQL, SQL Server, controladores ODBC ni LaTeX en el host.

En equipos ARM, incluido Apple Silicon, SQL Server usa `linux/amd64` mediante emulación. El primer arranque puede ser más lento.

4. Configuración segura

4.1. Archivo de entorno

El archivo `.env` contiene la configuración local y nunca debe enviarse a Git. Se crea desde el ejemplo:

```
cp .env.example .env
```

Se deben reemplazar todos los valores que empiecen con `ChangeMe_*`. Las claves deben cumplir la política de SQL Server: longitud suficiente y combinación de mayúsculas, minúsculas, números y símbolos.

Variable	Valor inicial	Uso
<code>COMPOSE_PROJECT_NAME</code>	<code>bi-ia-prototype</code>	Prefijo aislado de red, contenedores y volúmenes.
<code>FRONTEND_PORT</code>	<code>5173</code>	Puerto web de desarrollo; 8080 en la entrega.

Variable	Valor inicial	Uso
BACKEND_PORT	8000	Puerto de FastAPI y OpenAPI.
LOCAL_POSTGRES_PORT	55432	Publicación local de PostgreSQL.
LOCAL_SQLSERVER_PORT	51433	Publicación local de SQL Server.
POSTGRES_DB	bi_prototype	Base interna y futuro datamart.
POSTGRES_USER	bi_app	Cuenta de la aplicación interna.
POSTGRES_PASSWORD	secreto local	Clave de PostgreSQL.
SQLSERVER_DATABASE	AdventureWorks2022	Base externa restaurada.
SQLSERVER_USER	bi_reader	Cuenta exclusiva de lectura.
SQLSERVER_PASSWORD	secreto local	Clave del lector.
SQLSERVER_SA_PASSWORD	secreto local	Clave administrativa usada sólo durante inicialización.
IMAGE_PREFIX	ruta GHCR	Prefijo de las imágenes publicadas.
IMAGE_TAG	main	Entrega a ejecutar; se recomienda una etiqueta versionada.

Precaución. No copiar contraseñas, tokens, archivos `.env`, respaldos `.bak` ni volúmenes al repositorio. Ante una exposición, rotar inmediatamente la credencial y revisar el historial.

5. Instalación de desarrollo en este computador

El proyecto se encuentra en `/home/ceo/Proyectos/BI`. Para una primera ejecución:

```
cd "/home/ceo/Proyectos/BI"
cp .env.example .env
# Editar .env y cambiar todos los secretos ChangeMe_*
make bootstrap
```

`make bootstrap` realiza las siguientes acciones:

1. valida o crea el archivo de configuración local;
2. construye las imágenes propias;
3. crea una red y tres volúmenes exclusivos;
4. inicia PostgreSQL y espera su estado saludable;
5. inicia SQL Server, descarga y restaura AdventureWorks si el volumen está vacío;
6. crea y prueba el usuario `bi_reader`;
7. inicia FastAPI y después React;
8. espera hasta que los cuatro servicios respondan correctamente.

El primer arranque puede tardar varios minutos. Para observar la restauración:

```
docker compose logs -f sqlserver
```

6. Instalación en otro computador desde el código

6.1. Obtener el repositorio

El código puede clonarse sin invitación porque el repositorio es público. Para crear ramas en el repositorio central, revisar o fusionar cambios se requiere ser colaborador autorizado.

```
git clone https://github.com/LFXAS/prototipo-bi-ia.git
cd prototipo-bi-ia
```

La carpeta puede ubicarse en cualquier ruta; Compose utiliza rutas relativas.

6.2. Preparar secretos

```
cp .env.example .env
# Abrir .env y reemplazar todos los valores ChangeMe_*
```

Si alguno de los puertos está ocupado, cambiarlo en `.env` antes del arranque.

6.3. Construir y levantar

```
make bootstrap
```

6.4. Confirmar el estado

```
docker compose ps
curl http://localhost:8000/api/v1/health/live
curl http://localhost:8000/api/v1/health/ready
```

Las dos respuestas deben indicar `"status": ".ok"`; la segunda también debe indicar `"postgres": ".ok"`.

6.5. Abrir la aplicación

- Frontend: <http://localhost:5173>
- API: <http://localhost:8000>
- OpenAPI: <http://localhost:8000/docs>



Figura 1: Resultado esperado después de levantar el entorno de desarrollo.

7. Ejecución desde imágenes privadas de GHCR

Esta modalidad permite levantar una entrega sin compilar el código.

7.1. Autenticarse en GHCR

La persona debe tener acceso al repositorio y un token con permiso `read:packages`. El token se introduce por entrada estándar y no se escribe en el comando:

```
echo "$GHCR_TOKEN" | docker login ghcr.io -u USUARIO_GITHUB --password-stdin
```


Precaución. No guardar el token en el repositorio, en `.env` ni en capturas. La variable temporal debe eliminarse o cerrarse con la sesión.

7.2. Configurar la entrega

```
cp .env.release.example .env
```

En `.env`, establecer:

```
IMAGE_PREFIX=ghcr.io/LFXAS/prototipo-bi-ia  
IMAGE_TAG=main
```

Para una entrega académica estable, sustituir `main` por una etiqueta inmutable, por ejemplo `v1.0.0`.

7.3. Descargar e iniciar

```
docker compose -f compose.release.yaml pull  
docker compose -f compose.release.yaml up -d  
docker compose -f compose.release.yaml ps
```

En esta modalidad el frontend usa <http://localhost:8080> y la API conserva el puerto 8000.

8. Comprobación técnica

8.1. Comprobación rápida

```
make doctor
```

8.2. Comprobación completa

```
make verify
```

El control completo valida las dos variantes de Compose, construye y prueba frontend/backend y regenera toda la documentación PDF.

8.3. Contrato OpenAPI

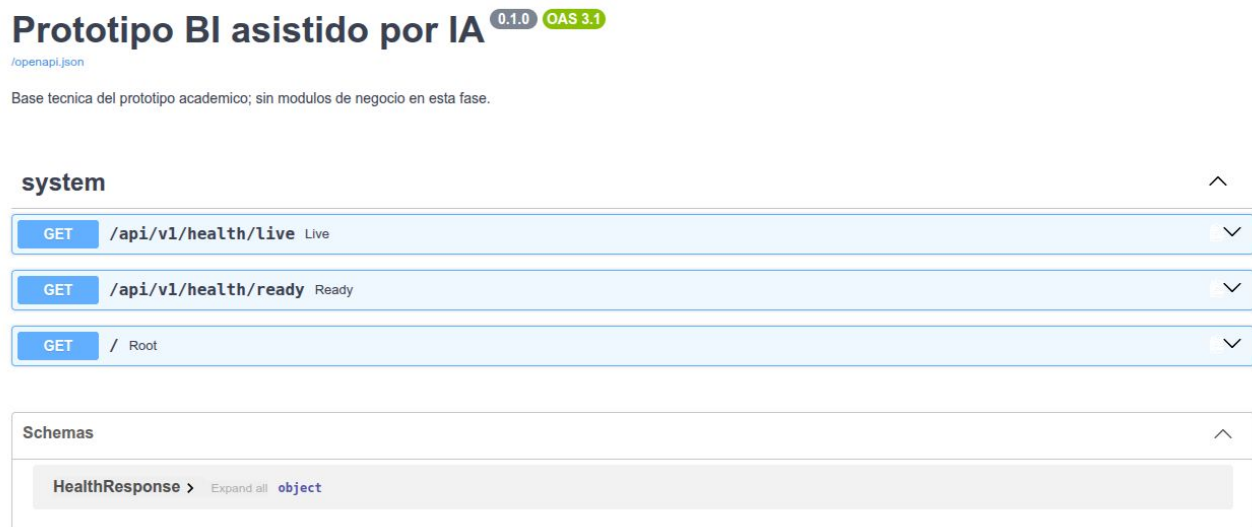


Figura 2: Endpoints técnicos disponibles en el Sprint 1.

8.4. Validación de AdventureWorks

La restauración correcta debe producir 71 tablas de usuario. La sonda del contenedor se autentica como `bi_reader`; por tanto, un estado saludable confirma que la cuenta puede conectar y leer.

Para una auditoría explícita, ingresar al contenedor usando variables ya inyectadas y ejecutar consultas controladas. No copiar las contraseñas al historial:

```
docker compose exec sqlserver bash
/opt/mssql-tools18/bin/sqlcmd -S localhost \
-U "$SQLSERVER_USER" -P "$SQLSERVER_PASSWORD" -C \
-d "$SQLSERVER_DATABASE" -Q \
"SELECT COUNT(*) AS tablas FROM sys.tables;"
exit
```

No se debe ejecutar una prueba destructiva sobre datos que deban conservarse. La automatización de inicialización ya verifica las denegaciones de escritura de forma controlada.

9. Operación diaria

Comando	Función
<code>make up</code>	Construye cambios y levanta el entorno en segundo plano.
<code>make down</code>	Detiene servicios y conserva datos.
<code>make ps</code>	Muestra servicios y estado de salud.
<code>make logs</code>	Sigue los registros recientes.
<code>make test</code>	Ejecuta pruebas en las etapas Docker de backend y frontend.
<code>make lint</code>	Ejecuta Ruff, Mypy y ESLint en contenedores.
<code>make compose-check</code>	Valida Compose de desarrollo y de entrega.
<code>make docs</code>	Regenera bitácora, informe de sprint y manual.
<code>make verify</code>	Ejecuta la verificación completa antes de entregar.

La recarga automática de Vite y FastAPI permite editar código sin instalar sus dependencias en el host.

10. Persistencia y reinicio de datos

10.1. Detener sin borrar

```
docker compose down
```

Los volúmenes permanecen y el siguiente arranque reutiliza PostgreSQL y AdventureWorks.

10.2. Reconstruir desde cero

Precaución. El siguiente procedimiento elimina de forma irreversible sólo los volúmenes del proyecto Compose actual. Confirmar que no contienen resultados que deban conservarse.

```
docker compose down --volumes
make bootstrap
```

La restauración de AdventureWorks volverá a descargar el respaldo cuando el volumen esté vacío.

10.3. Qué se versiona

- Se versionan Dockerfiles, Compose, scripts, migraciones, documentación y datos sintéticos aprobados.
- No se versionan volúmenes, contraseñas, tokens, archivos `.bak`, `.dump` ni datos productivos.
- AdventureWorks se reconstruye desde el respaldo oficial y PostgreSQL desde scripts/migraciones.

11. Actualización y reversión

11.1. Entorno construido desde código

```
git pull --ff-only
docker compose up --build -d
make doctor
```

11.2. Entorno basado en GHCR

```
docker compose -f compose.release.yaml pull
docker compose -f compose.release.yaml up -d
```

Para volver a una versión conocida, fijar **IMAGE_TAG** a una etiqueta anterior y repetir **pull/up**. No usar **latest** para entregas evaluables.

Cuando existan migraciones de datos, la reversión deberá seguir el procedimiento de la versión específica. Nunca sustituir una migración por la copia manual de un volumen.

12. Flujo Git y CI/CD

12.1. Desarrollo por rama

La rama predeterminada **develop** integra el trabajo diario y **main** conserva únicamente entregas estables. Las ramas de trabajo parten de **develop** con los prefijos **feature/**, **fix/**, **docs/** o **chore/**.

```
git switch develop
git pull --ff-only origin develop
git switch -c feature/nombre-cambio
# editar y comprobar
make verify
git add .
git commit -m "feat: descripcion breve"
git push -u origin feature/nombre-cambio
gh pr create --base develop --fill
```

El pull request hacia **develop** exige CI aprobada, una revisión y conversaciones resueltas. Cuando la iteración sea estable, se abre un segundo pull request de **develop** hacia **main**; esa promoción vuelve a ejecutar CI y, al fusionarse, activa CD. El procedimiento completo está en **docs/git-workflow.md**.

Las ramas **develop** y **main** tienen protección activa: impiden force-push y eliminación, exigen pull request, una aprobación, conversaciones resueltas y ocho controles de CI con la rama actualizada. La excepción administrativa se reserva para recuperación y debe justificarse en la bitácora.

12.2. Integración continua

En cada push y pull request hacia **develop** o **main**, CI:

1. valida que los PR de trabajo lleguen a **develop** y que sólo **develop** promueva a **main**;
2. valida el archivo Compose;
3. construye la etapa de prueba del backend;

4. construye la etapa de prueba del frontend;
5. construye las imágenes de infraestructura;
6. compila los documentos LaTeX y verifica que los PDF versionados estén actualizados.

12.3. Entrega continua

Después de un cambio en `main` o una etiqueta `v*`, CD valida y publica:

- `prototipo-bi-ia-backend`;
- `prototipo-bi-ia-frontend`;
- `prototipo-bi-ia-postgres`;
- `prototipo-bi-ia-sqlserver`;
- `prototipo-bi-ia-docs`.

Las ejecuciones de línea base CI 32749237009 y CD 32749237059 finalizaron exitosamente para el commit `ddf2551`.

Dependabot trabaja contra la rama predeterminada `develop`. Las actualizaciones npm menores y parches se agrupan; los saltos mayores se revisan de forma intencional y separada para evitar mezclar migraciones incompatibles.

13. Seguridad operativa

1. Cambiar todas las credenciales de ejemplo antes del primer arranque.
2. Mantener `.env` fuera de Git y fuera de los documentos.
3. Usar `bi_reader` para AdventureWorks; no conectar la aplicación como `sa`.
4. Conservar `SQLSERVER_APPLICATION_INTENT=ReadOnly`, pero apoyarse en permisos reales del motor.
5. Gestionar la visibilidad de cada paquete GHCR de forma independiente y no incluir secretos ni datos reales en ninguna imagen.
6. Otorgar acceso únicamente a cuentas identificadas y revocar colaboradores que ya no participen.
7. Revisar los cambios de Dependabot y no fusionarlos sin CI aprobada.
8. Cuando se implemente RBAC, FastAPI debe autorizar cada endpoint; ocultar menús en React no es un control de seguridad.

14. Diagnóstico de fallos

Síntoma	Causa probable	Acción recomendada
Puerto ocupado	Otro servicio usa 5173, 8000, 55432 o 51433.	Cambiar el puerto correspondiente en <code>.env</code> y reiniciar Compose.
Backend vivo, pero ready falla	PostgreSQL no está saludable o las credenciales no coinciden.	Revisar <code>docker compose ps</code> y los registros de PostgreSQL/backend.
SQL Server no queda saludable	Memoria insuficiente, clave SA inválida o descarga bloqueada.	Asignar 8 GB a Docker, corregir la clave y revisar los registros.
Restauración muy lenta	Primera descarga o emulación ARM.	Esperar y seguir <code>docker compose logs -f sqlserver</code> .
SQL Server queda no saludable tras reiniciar	AdventureWorks todavía está recuperándose.	Esperar la sonda; el inicializador comprueba la base antes de recrear la marca de disponibilidad.
Frontend no muestra API disponible	Backend no saludable o proxy mal configurado.	Probar <code>/health/live</code> , revisar puertos y registros del frontend.
GHCR devuelve denied	Falta autenticación, acceso al paquete o permiso <code>read:packages</code> .	Repetir <code>docker login ghcr.io</code> con una cuenta autorizada.
PDF no se actualiza	No se regeneró después de editar LaTeX.	Ejecutar <code>make docs</code> y revisar los archivos modificados.
CI rechaza el cambio	Pruebas, formato, Compose o PDF desactualizado.	Abrir el trabajo fallido, reproducir con <code>make verify</code> y corregir antes de fusionar.

14.1. Registros útiles

```
docker compose logs --tail=200 backend
docker compose logs --tail=200 frontend
docker compose logs --tail=200 postgres
docker compose logs --tail=200 sqlserver
```

15. Lista de comprobación para una demostración

1. Confirmar que el commit o etiqueta corresponde a la entrega evaluada.
2. Ejecutar `make verify` y conservar la evidencia de CI.
3. Ejecutar `docker compose ps`; los cuatro servicios deben estar saludables.
4. Consultar `/health/live` y `/health/ready`.
5. Abrir frontend y OpenAPI en el navegador.
6. Confirmar que AdventureWorks contiene 71 tablas y el acceso externo usa `bi_reader`.
7. Confirmar que no hay secretos ni archivos `.env` en los cambios de Git.
8. Registrar SHA, etiqueta y digest de imágenes en la bitácora.
9. Conservar los PDF generados para la evidencia de tesis.

16. Mantenimiento de la documentación

Los documentos se compilan dentro de la imagen `bi-ia-docs`; no se instala LaTeX en el host.

```
make logbook
make sprint-report
make technical-manual
make docs
```

Después de cambios técnicos se deben actualizar, según corresponda, README, arquitectura, guía de réplica, manual técnico, informe del sprint y bitácora. Los PDF finales se versionan; los archivos auxiliares de LaTeX se ignoran.

17. Responsabilidades y escalamiento

Rol	Responsabilidad
Autoras	Aprobar alcance, secretos, accesos, datos de prueba y entregas académicas.
Colaborador técnico	Trabajar por rama, ejecutar pruebas, actualizar documentación y solicitar revisión.
CI/CD	Repetir controles y publicar imágenes después de la integración autorizada.
FastAPI futuro	Aplicar autorización real, validaciones determinísticas y auditoría.
Responsable de datos	Confirmar que la fuente externa permanezca en lectura y que no se incorporen datos productivos.

Ante pérdida de datos locales, credenciales expuestas, imágenes inesperadas o permisos de escritura en AdventureWorks, detener la demostración, conservar registros, revocar las credenciales afectadas y documentar la resolución antes de continuar.

A. Referencia rápida de URLs y puertos

Recurso	Dirección	Observación
Frontend desarrollo	http://localhost:5173	React/Vite
Frontend entrega	http://localhost:8080	Nginx/GHCR
API	http://localhost:8000	FastAPI
OpenAPI	http://localhost:8000/docs	Contrato interactivo
Salud básica	http://localhost:8000/api/v1/health/live	Proceso de API
Preparación	http://localhost:8000/api/v1/health/ready	API y PostgreSQL
PostgreSQL	<code>localhost:55432</code>	Acceso técnico local
SQL Server	<code>localhost:51433</code>	Acceso técnico local

B. Recuperación mínima

Si el entorno queda en un estado desconocido y no existen datos que conservar:

```
cd "/ruta/del/repositorio"
docker compose down --volumes
docker compose build --no-cache
make bootstrap
make verify
```

Precaución. La recuperación elimina los volúmenes del proyecto actual. No ejecutarla como diagnóstico rutinario ni en una carpeta cuya identidad Compose no haya sido confirmada.

C. Control del manual

Campo	Valor
Estado	Primera edición técnica, sujeta a revisión de las autoras y del tutor.
Custodia	Repositorio público LFXAS/prototipo-bi-ia; escritura restringida a colaboradores.
Audiencia	Autoras, tutor, revisores, colaboradores y lectores académicos.
Actualización	Debe acompañar todo cambio de instalación, puertos, secretos, imágenes, pruebas o recuperación.
Fuente editable	docs/manual-tecnico/manual-tecnico.tex.
Compilación	make technical-manual o make docs.

Historial de cambios

Versión	Fecha	Cambio
1.0.0	24-08-2026	Primera versión: instalación, réplica, operación, CI/CD, seguridad, diagnóstico y recuperación del ambiente del Sprint 1.
1.1.0	24-08-2026	Flujo feature/* hacia develop , promoción a main , protecciones y política de Dependabot.

Registro de revisión

Rol	Nombre y conformidad	Fecha
Autora	_____	_____
Autora	_____	_____
Tutor/revisor	_____	_____

Las firmas formales pertenecen a la tesis entregada y no se versionan si contienen datos personales cuya distribución no fue autorizada.